AI/ML FRAMEWORKS

scikit-learn

Master the consistent estimator API, pipelines, classic models, and rigorous model evaluation that make scikit-learn the backbone of classical machine learning.

scikit-learn machine-learning python pipelines cross-validation model-selection preprocessing

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What scikit-learn Is and Why It Matters
2. 2. Getting Started
3. 3. The Estimator API: fit, predict, transform
4. 4. Splitting Data Honestly
5. 5. Preprocessing and Feature Engineering
6. 6. Pipelines and ColumnTransformer
7. 7. Classic Models: A Tour
8. 8. Metrics: Measuring What Matters
9. 9. Cross-Validation and Model Selection
10. 10. A Worked Mini-Example: Titanic Survival
11. 11. Cheat-Sheet Reference
12. 12. Best Practices and Common Pitfalls
13. 13. Interview Questions & Answers
14. 14. Further Learning

scikit-learn is the de-facto standard library for classical machine learning in Python. Its genius is one tiny, uniform contract — `fit`, `predict`, `transform` — shared by hundreds of algorithms. Learn the contract once and you can swap a logistic regression for a random forest by changing a single line, chain preprocessing and models into leak-proof pipelines, and tune everything with the same grid-search machinery.

1. What scikit-learn Is and Why It Matters

scikit-learn (imported as `sklearn`) sits on top of NumPy, SciPy, and (optionally) pandas. It is **not** a deep-learning framework — there is no GPU autodiff. Its sweet spot is *tabular data*: the structured, columnar datasets that dominate real-world business problems — fraud detection, churn, pricing, risk scoring, demand forecasting. For these, gradient-boosted trees and linear models usually beat neural nets, and scikit-learn ships all of them behind one API.

CONSISTENCY

Every algorithm is an *estimator* with the same methods. Knowledge transfers instantly across models.

COMPOSABILITY

`Pipeline` and `ColumnTransformer` glue preprocessing to models so the whole flow is one object you can fit, score, and persist.

RIGOR

First-class cross-validation, metrics, and hyper-parameter search make honest evaluation the default, not an afterthought.

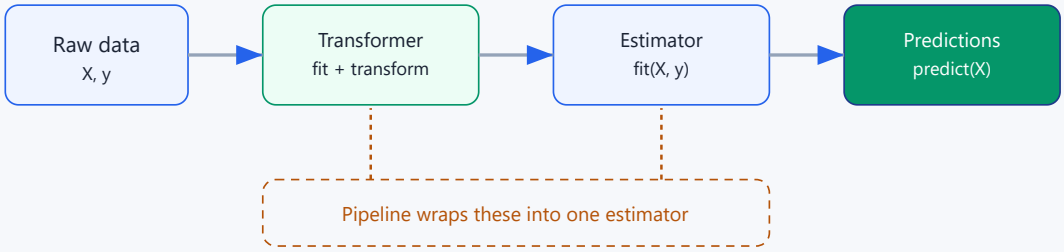


Fig 1 — The core mental model: transformers reshape features, estimators learn, and a Pipeline chains them into one fit/predict object.

> **Mental model**
> Think of scikit-learn as LEGO. Each brick (estimator) has the same studs (`fit` / `predict` / `transform`). You never glue bricks permanently — you snap them together into a Pipeline and can pull any brick out and replace it.

2. Getting Started

Install into an isolated environment. The `scikit-learn` package name differs from the import name `sklearn`.

BASH

```bash
# Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# Install scikit-learn plus the usual companions
pip install scikit-learn pandas matplotlib

# Verify
python -c "import sklearn; print(sklearn.__version__)"   # 1.7.x as of 2026
```

A complete first program — load data, split, fit, evaluate — in a dozen lines:

PYTHON

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)          # X: (150, 4) features, y: (150,) labels

# Hold out 25% for honest evaluation; stratify keeps class ratios balanced
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

clf = RandomForestClassifier(n_estimators=200, random_state=42)
clf.fit(X_train, y_train)                  # learn from training data only
preds = clf.predict(X_test)                # predict on unseen data

print(f"accuracy = {accuracy_score(y_test, preds):.3f}")   # ~0.95
```

✦ **Reproducibility**

Always pass `random_state` to any object that uses randomness (splits, forests, KMeans). Without it, results change every run and bugs become impossible to reproduce.

3. The Estimator API: fit, predict, transform

Everything in scikit-learn implements a subset of four verbs. This uniformity is the whole point of the library.

Method	Who implements it	What it does
`fit(X, y=None)`	Every estimator	Learns parameters from data. Stores results as attributes ending in `_` (e.g. `coef_`). Returns `self`.
`predict(X)`	Classifiers, regressors, clusterers	Produces a label or value for each row of X.
`transform(X)`	Transformers (scalers, encoders, PCA)	Returns a modified feature matrix.
`fit_transform(X)`	Transformers	Convenience: fit then transform in one call (often optimized).
`predict_proba(X)`	Most classifiers	Per-class probabilities, shape (n_samples, n_classes).
`score(X, y)`	Classifiers, regressors	Default metric: accuracy for classifiers, R^2 for regressors.

⚠ **The golden rule**
Call `fit` (or `fit_transform`) on **training data only**. On test data call `transform` and `predict` only. Fitting a scaler or encoder on the full dataset leaks test-set statistics into training — the single most common cause of "great in notebook, terrible in production."

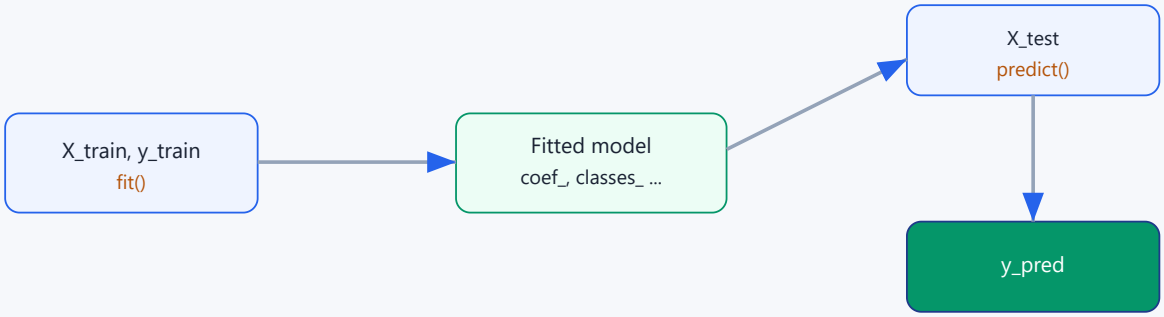


Fig 2 — Learned state lives on the fitted estimator as trailing-underscore attributes; predict reuses that state on new data.

4. Splitting Data Honestly

The cardinal sin of ML is evaluating on data the model has seen. `train_test_split` is the first defense; cross-validation (Section 9) is the stronger one.

PYTHON

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,       # 20% held out (or train_size=0.8)
    random_state=42,     # reproducible shuffle
    stratify=y,          # preserve class proportions (classification)
    shuffle=True,        # default; set False for time series!
)
```

Time-series trap

Never randomly shuffle time-ordered data — you would train on the future to predict the past. Use `TimeSeriesSplit` or a manual cutoff date instead.

5. Preprocessing and Feature Engineering

Raw features are rarely model-ready. Numeric columns need scaling for distance- and gradient-based models; categorical columns need encoding; missing values need imputing. All of these are *transformers*.

Transformer	Use for	Notes
`StandardScaler`	Center to mean 0, scale to std 1	Needed by SVM, KNN, logistic regression, PCA, KMeans.
`MinMaxScaler`	Squash into [0, 1]	Good when you need bounded features.
`RobustScaler`	Scale using median/IQR	Resistant to outliers.
`OneHotEncoder`	Nominal categoricals	`handle_unknown='ignore'` for unseen categories at predict time.
`OrdinalEncoder`	Ordered categoricals	Only when order is meaningful (low < mid < high).
`SimpleImputer`	Fill missing values	`strategy='median'` is a robust default for numerics.

PYTHON

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # learns mean_ and scale_ HERE
X_test_scaled  = scaler.transform(X_test)        # reuses TRAIN statistics

print(scaler.mean_)    # learned per-column means
# Tree-based models (RF, gradient boosting) do NOT need scaling —
# they split on thresholds, so monotonic rescaling changes nothing.
```

Tree models are scale-invariant; distance and gradient models are not. Keep this table in your head:

NEEDS SCALING

SVM, KNN, Logistic/Linear Regression (for regularization), PCA, KMeans, neural nets.

SCALE-INVARIANT

Decision Trees, Random Forests, Gradient Boosting, Extra Trees.

6. Pipelines and ColumnTransformer

A `Pipeline` chains transformers and a final estimator into a single object. Calling `fit` on the pipeline fits each step in order; calling `predict` applies every transform then the model. This makes leakage structurally impossible during cross-validation, because the scaler is re-fit on each training fold only.

PYTHON

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000)),
])

pipe.fit(X_train, y_train)      # scaler.fit_transform → clf.fit
pipe.predict(X_test)            # scaler.transform      → clf.predict
pipe.score(X_test, y_test)
```

Real datasets mix numeric and categorical columns needing different treatment. `ColumnTransformer` routes each column subset to the right transformer, then concatenates the results.

PYTHON

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

num_cols = ["age", "income", "balance"]
cat_cols = ["country", "product"]

numeric = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
categorical = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore")),
])

pre = ColumnTransformer([
    ("num", numeric, num_cols),
    ("cat", categorical, cat_cols),
])

model = Pipeline([
    ("pre", pre),
    ("clf", RandomForestClassifier(n_estimators=300, random_state=42)),
])
model.fit(X_train, y_train)     # X_train is a DataFrame with the named columns
```

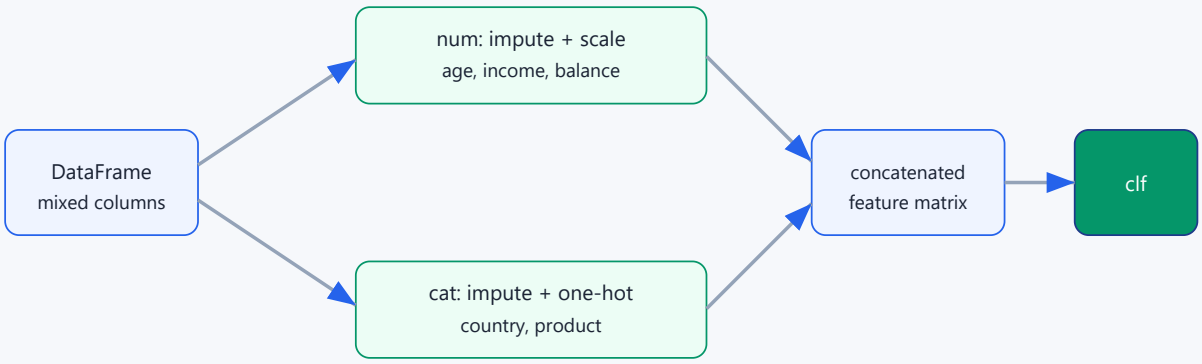


Fig 3 — ColumnTransformer splits columns by type, transforms each subset in parallel, concatenates, then feeds the final estimator.

✓ **Why this matters**

One `model` object now encapsulates imputation, scaling, encoding, and the classifier. Persist it with `joblib.dump` and production inference is a single `model.predict(df)` — no risk of forgetting a preprocessing step.

7. Classic Models: A Tour

Swap any model below into the pipeline above by changing the final step. The API is identical.

Linear and Logistic Regression

PYTHON

```python
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge

# Regression: predict a continuous value
reg = LinearRegression().fit(X_train, y_train)
print(reg.coef_, reg.intercept_)        # learned weights and bias

# Ridge adds L2 regularization (alpha controls strength) to fight overfitting
ridge = Ridge(alpha=1.0).fit(X_train, y_train)

# Classification despite the name: logistic regression outputs probabilities
log = LogisticRegression(C=1.0, max_iter=1000).fit(X_train, y_train)
log.predict_proba(X_test)[:5]           # per-class probabilities
```

ℹ **C vs alpha**

In `LogisticRegression` / `SVC`, `C` is *inverse* regularization strength — smaller C = more regularization. In `Ridge` / `Lasso`, `alpha` is direct — larger alpha = more regularization. Easy to mix up.

Decision Trees, Random Forests, Gradient Boosting

PYTHON

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, HistG

# A single tree: interpretable but prone to overfitting
tree = DecisionTreeClassifier(max_depth=5, random_state=42).fit(X_train, y_train)

# Random Forest: many de-correlated trees voting — strong, robust default
rf = RandomForestClassifier(n_estimators=300, max_depth=None,
                            n_jobs=-1, random_state=42).fit(X_train, y_train)
print(rf.feature_importances_)           # which features drove splits

# HistGradientBoosting: fast, handles NaNs natively — top tabular performer
hgb = HistGradientBoostingClassifier(max_iter=300,
                                     learning_rate=0.1).fit(X_train, y_train)
```

SVM, KNN, KMeans

PYTHON

```python
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans

# SVM with RBF kernel — powerful on small/medium data; MUST scale features first
svm = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True)

# KNN — lazy learner; prediction compares to k nearest training points
knn = KNeighborsClassifier(n_neighbors=5, weights="distance")

# KMeans — UNSUPERVISED: no y. Groups data into k clusters.
km = KMeans(n_clusters=3, n_init="auto", random_state=42).fit(X_train)
print(km.labels_, km.cluster_centers_)   # assignments + centroids
new_labels = km.predict(X_test)          # assign new points to nearest centroid
```

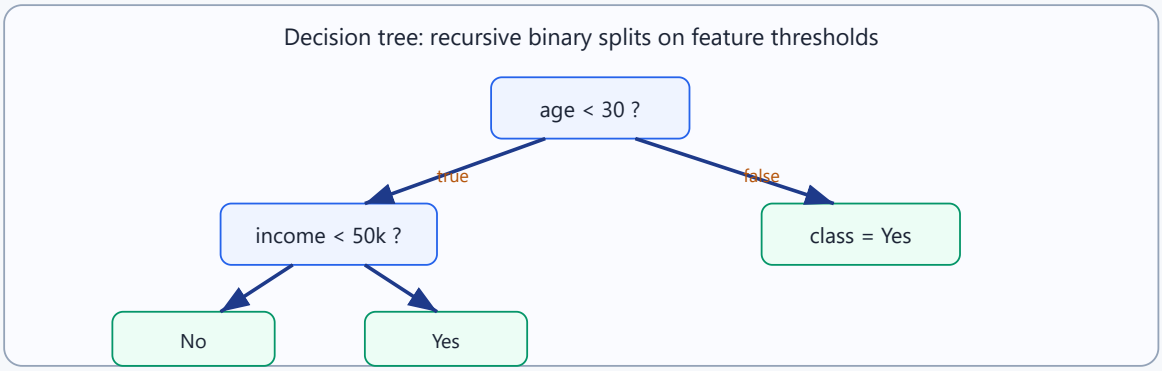


Fig 4 — A decision tree splits the feature space with axis-aligned thresholds; a Random Forest averages hundreds of such trees.

8. Metrics: Measuring What Matters

Accuracy lies on imbalanced data. If 99% of transactions are legitimate, a model that predicts "legit" always scores 99% accuracy and catches zero fraud. Choose the metric that reflects the business cost.

Task	Metric	When to use
Classification	`accuracy_score`	Balanced classes, equal error costs.
Classification	`precision_score`	Cost of false positives is high (spam filter).
Classification	`recall_score`	Cost of false negatives is high (cancer screening).
Classification	`f1_score`	Need a balance of precision and recall.
Classification	`roc_auc_score`	Ranking quality, threshold-independent.
Regression	`mean_absolute_error`	Robust, same units as target.
Regression	`root_mean_squared_error`	Penalizes large errors more.
Regression	`r2_score`	Fraction of variance explained (1.0 = perfect).

PYTHON

```python
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, root_mean_squared_error)

y_pred = clf.predict(X_test)
print(confusion_matrix(y_test, y_pred))        # rows=true, cols=predicted
print(classification_report(y_test, y_pred))   # precision/recall/f1 per class

# For probabilistic ranking (binary), use predicted probabilities, not labels
proba = clf.predict_proba(X_test)[:, 1]
print("AUC:", roc_auc_score(y_test, proba))
```

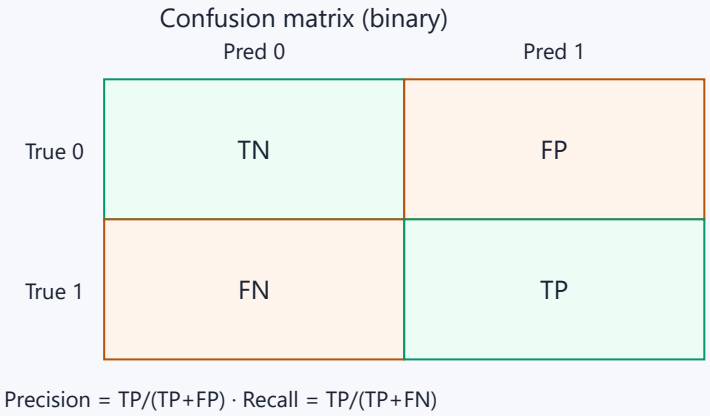


Fig 5 — Precision asks "of those I flagged, how many were right?"; recall asks "of the real positives, how many did I catch?"

9. Cross-Validation and Model Selection

A single train/test split gives a noisy estimate. **K-fold cross-validation** rotates the test fold across the data, training k times, so every row is tested exactly once. The mean score is far more trustworthy.

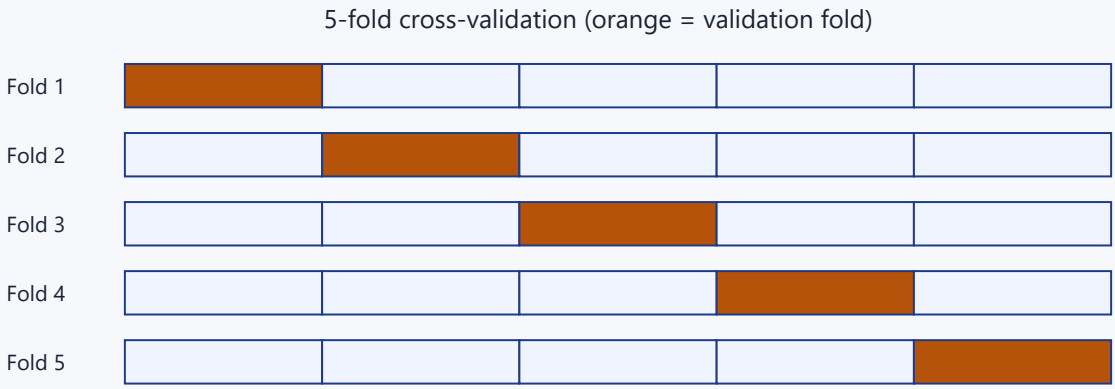


Fig 6 — Each row trains on blue blocks and validates on the orange one; averaging the 5 scores yields a stable estimate.

PYTHON

```python
from sklearn.model_selection import cross_val_score, cross_validate, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Pass the PIPELINE, not a bare model — scaler is re-fit per fold (no leak)
scores = cross_val_score(pipe, X, y, cv=cv, scoring="f1_macro")
print(f"{scores.mean():.3f} +/- {scores.std():.3f}")

# cross_validate returns multiple metrics and fit/score times
results = cross_validate(pipe, X, y, cv=cv,
                         scoring=["accuracy", "roc_auc"], return_train_score=True)
```

Hyper-parameter tuning

`GridSearchCV` exhaustively tries every combination with cross-validation; `RandomizedSearchCV` samples a fixed budget — better when the grid is large. Use `step__param` double-underscore syntax to reach inside pipeline steps.

PYTHON

```python
from sklearn.model_selection import GridSearchCV

param_grid = {
    "clf__C": [0.01, 0.1, 1, 10],          # 'clf' is the pipeline step name
    "clf__penalty": ["l1", "l2"],
    "clf__solver": ["liblinear"],
}

search = GridSearchCV(pipe, param_grid, cv=5,
                      scoring="f1_macro", n_jobs=-1, refit=True)
search.fit(X_train, y_train)

print(search.best_params_, search.best_score_)
best_model = search.best_estimator_        # already refit on all training data
best_model.predict(X_test)
```

> ⚠ **Nested leakage**
>
> If you tune hyper-parameters with CV and then report that same CV score as your final performance, you have optimistically biased it. For an unbiased estimate use *nested* CV, or keep a final untouched hold-out test set you only score once.

10. A Worked Mini-Example: Titanic Survival

End-to-end: mixed-type preprocessing, a tuned classifier, honest evaluation — the shape of nearly every real project.

PYTHON

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import classification_report

df = pd.read_csv("titanic.csv")
y = df["Survived"]
X = df[["Pclass", "Sex", "Age", "SibSp", "Fare", "Embarked"]]

num = ["Age", "SibSp", "Fare"]
cat = ["Pclass", "Sex", "Embarked"]

pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("ohe", OneHotEncoder(handle_unknown="ignore"))]), cat),
])

model = Pipeline([("pre", pre),
                  ("clf", HistGradientBoostingClassifier(random_state=42))])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state=42)

grid = {"clf__learning_rate": [0.05, 0.1], "clf__max_iter": [200, 400]}
search = GridSearchCV(model, grid, cv=5, scoring="roc_auc", n_jobs=-1)
search.fit(X_tr, y_tr)

print("Best CV AUC:", round(search.best_score_, 3))
print(classification_report(y_te, search.predict(X_te)))

# Persist the entire pipeline for production
import joblib
joblib.dump(search.best_estimator_, "titanic_model.joblib")
```

11. Cheat-Sheet Reference

Goal	Import	Call
Split data	`model_selection.train_test_split`	`train_test_split(X, y, stratify=y)`
Scale features	`preprocessing.StandardScaler`	`.fit_transform(X_train)`
Encode categories	`preprocessing.OneHotEncoder`	`handle_unknown='ignore'`
Chain steps	`pipeline.Pipeline`	`Pipeline([(name, step), ...])`
Route columns	`compose.ColumnTransformer`	`[(name, transformer, cols)]`
Cross-validate	`model_selection.cross_val_score`	`cross_val_score(est, X, y, cv=5)`
Tune params	`model_selection.GridSearchCV`	`step__param` keys
Save model	`joblib`	`joblib.dump(est, 'm.joblib')`

MODEL COMPARISON

Model	Strengths	Watch out for
Logistic Regression	Fast, interpretable, calibrated probabilities	Only linear boundaries; needs scaling
Random Forest	Strong default, no scaling, feature importance	Large memory; weaker on extrapolation
HistGradientBoosting	Top tabular accuracy, handles NaNs	More hyper-parameters to tune
SVM (RBF)	Powerful on small/medium data	Scales poorly past ~50k rows; must scale
KNN	Simple, no training phase	Slow predict; curse of dimensionality
KMeans	Fast unsupervised clustering	Must pick k; assumes spherical clusters

12. Best Practices and Common Pitfalls

✓ **Do**

Wrap all preprocessing in a Pipeline so transforms are fit per CV fold. Always stratify classification splits. Set `random_state` everywhere. Keep one final hold-out set you touch exactly once.

✦ **Do**

Start with a dumb baseline (`DummyClassifier` / `DummyRegressor`). If your fancy model barely beats "predict the majority class," something is wrong. Prefer `n_jobs=-1` to use all cores for forests and grid search.

⚠ **Avoid**

Fitting scalers/encoders on the whole dataset before splitting (data leakage). Using accuracy on imbalanced data. Shuffling time-series. Comparing models on different random splits.

✕ **Never**

Report your cross-validation / grid-search score as the model's real-world performance after you used that same CV to choose hyper-parameters — it is optimistically biased. The number stakeholders see must come from data the selection process never saw.

13. Interview Questions & Answers

What is the difference between `fit`, `transform`, and `fit_transform`?

`fit` learns parameters from data (e.g. a scaler's mean and std) and stores them; `transform` applies those learned parameters to produce new output; `fit_transform` does both in one step and is used on *training* data only. On test data you call `transform` alone so you reuse the training statistics and avoid leakage.

What is data leakage and how do pipelines prevent it?

Leakage is when information from outside the training fold influences the model — most commonly by fitting a scaler or imputer on the full dataset. A `Pipeline` prevents it because during cross-validation each fold calls `fit` on only that fold's training portion, so preprocessing statistics never see the validation data.

Why is accuracy a poor metric for imbalanced classification?

With 99% negatives, always predicting "negative" scores 99% accuracy while catching zero positives. Precision, recall, F1, and ROC-AUC reveal performance on the minority class, which is usually the class you actually care about.

Explain precision versus recall.

Precision = TP/(TP+FP): of everything predicted positive, how many were truly positive — penalizes false alarms. Recall = TP/(TP+FN): of all actual positives, how many were caught — penalizes misses. There is a trade-off: lowering the decision threshold raises recall but lowers precision. F1 is their harmonic mean.

When should you scale features, and which models need it?

Scale when the algorithm depends on distances or gradient magnitudes: SVM, KNN, KMeans, PCA, and regularized linear models. Tree-based models (decision trees, random forests, gradient boosting) are scale-invariant because they split on thresholds, so monotonic rescaling does not change the splits.

How does Random Forest reduce overfitting compared to a single decision tree?

It trains many trees on bootstrap samples (bagging) and considers only a random subset of features at each split. This de-correlates the trees; averaging their votes cancels much of the variance of any single deep tree, yielding lower generalization error without materially increasing bias.

What does `GridSearchCV` do, and what is the `__` syntax?

It performs an exhaustive search over a parameter grid, scoring each combination with k-fold cross-validation, then refits the best estimator on all training data. The double-underscore (`step__param`) addresses a parameter of a named step inside a Pipeline, e.g. `clf__C` sets `C` on the step named `clf`.

What is the difference between GridSearchCV and RandomizedSearchCV?

GridSearch tries every combination — exhaustive but exponential in the number of parameters. RandomizedSearch samples a fixed number of random combinations (`n_iter`), which explores large or continuous spaces far more efficiently and often finds comparable optima for a fraction of the compute.

Why pass `stratify=y` to `train_test_split`?

It preserves the class proportions of the full dataset in both splits. Without it, a random split can under-represent a minority class in the test set (or omit it entirely), making the evaluation unreliable, especially for imbalanced problems.

What are the trailing-underscore attributes like `coef_` or `classes_`?

By convention, attributes learned during `fit` end with an underscore. They did not exist before fitting. Examples: `coef_` (linear weights), `feature_importances_` (trees), `cluster_centers_` (KMeans), `classes_` (label ordering). Accessing them before `fit` raises `NotFittedError`.

How do supervised and unsupervised estimators differ in the API?

Supervised estimators take labels: `fit(X, y)` and `predict(X)`. Unsupervised ones like KMeans or PCA take only features: `fit(X)`. Clusterers expose `labels_` after fitting and can `predict` new points; dimensionality reducers expose `transform`.

Logistic Regression is a classifier — why "regression" in the name?

It regresses the log-odds of the positive class as a linear function of the features, then passes the result through a sigmoid to produce a probability. The underlying model is a linear regression on log-odds; thresholding that probability turns it into a classifier.

14. Further Learning

- **Official User Guide** — scikit-learn.org/stable/user_guide.html is unusually well-written; read the sections on Pipelines and Model Selection in full.
- **Gradient boosting libraries** — graduate to XGBoost, LightGBM, and CatBoost; they share scikit-learn's API and dominate tabular competitions.
- **Model inspection** — learn `permutation_importance`, partial dependence plots, and SHAP for explaining predictions.
- **Imbalanced data** — explore `imbalanced-learn` (SMOTE, class weights) for fraud/anomaly problems.
- **Calibration** — study `CalibratedClassifierCV` when you need trustworthy probabilities, not just rankings.
- **Deployment** — practice persisting pipelines with `joblib` and serving them behind a FastAPI endpoint.